

B.2 QUALITY METRICS

Moreover, we evaluate all models on established image and video generation metrics, namely, FID (Heusel et al., 2017), FVD (Unterthiner et al., 2018), and CLIPSIM (Wu et al., 2021). We provide results for RealEstate10K (Zhou et al., 2018) and MSR-VTT (Xu et al., 2016) in Tab. 4.

It is important to highlight that FID and FVD are not direct measurements of visual quality—these metrics measure similarities between dataset distributions. So while fine-tuning on a dataset will degrade these scores relative to the original model, this degradation is expected because the model is fitting a different data distribution than the one used in the original training run. The same degradation in FID after fine-tuning is observed in the original ControlNet paper (Zhang et al., 2023), where the authors report an increase in FID when comparing the StableDiffusion base model (6.09 FID) to their fine-tuned ControlNet model (15.27 FID; see their Table 3). We believe that joint training with 2D video data may help to alleviate this degradation, and we will explore joint training strategies as part of future work.

B.3 BASELINE VARIANTS

We provide further variants of MotionCtrl and CameraCtrl adopted to the SnapVideo base model. These variants performed worse than the ones presented in the main paper, hence we mainly show them here for completeness to show the different implementations we explored.

On top of that, we train a variant of MotionCtrl where instead of integrating the camera matrices into the context vector, we integrate them into the output of the cross-attention layer between the latent tokens and patches (as in our approach). We observe degraded camera control compared to our approach. Moreover, we train a variant of CameraCtrl where we incorporate the Plucker embeddings of the CameraCtrl camera encoder into the patches instead of the latents. We observe worse camera control compared to our model. Furthermore, we train a variant of MotionCtrl where we freeze all base model layers similar to CameraCtrl and our approach. We use the same batch size, same number of iterations, and same parameter size as our proposed method for fair comparisons. Note that MotionCtrl unfreezes the attention layer after injecting camera features into the base model. This experiment highlights that we outperform MotionCtrl independent of training or freezing these attention layers. We show results for camera pose accuracy, multi-view generation, and video quality in Tab. 5, Tab. 6, and Tab. 7 respectively.

B.4 GENERALIZATION OF CAMERA TRAJECTORIES

We conduct additional experiments with horizontal and vertical panning, where the camera trajectory is defined by rotation-only camera matrices without any translation. Specifically, we manually construct each trajectory by randomly selecting either the x-, y-, or z-axis and randomly sampling the angular extent of the trajectory (from 0 to 120 degrees). We set the camera translation to the zero vector. We show results in Tab. 8 and observe higher accuracy for our methods. Furthermore, we include results for non-random, user-defined camera trajectories that involve camera movements with significant directional changes including both rotations and translations in Tab. 9. These include following trajectories: rotation around clockwise; rotation around anticlockwise; rotation clockwise without translation; rotation anticlockwise without translation; zoom out, then up translation; translation right, then rotation anticlockwise; translation left, then rotation clockwise; translation left; translation right; translation up; translation down. We observe that our method generalizes to input camera trajectories with variable rotations and translations.

B.5 EXPERIMENTS WITH VANILLA DiT

Instead of building upon the FIT (Chen & Li, 2023) architecture, we implemented our VD3D method on top of a pre-trained text-to-video DiT model (Peebles & Xie, 2023) in the latent space of the CogVideoX (Yang et al., 2024d) autoencoder. We include these results in Tab. 10 and Tab. 11. Instead of building on top of read attention in FIT, we incorporate the ControlNet conditioning on top of the vanilla attention mechanism of the actual tokens. We observe that the vanilla DiT version further improves quality and camera accuracy on out-of-distribution prompts (MSR-VTT). We believe that

our proposed method of spatio-temporal Plucker tokens and aligning them with video patch tokens through a ControlNet-type of conditioning mechanism is agnostic to the transformer architecture and will serve as a starting point for follow-up works.

Base video DiT architecture details. The video DiT architecture follows the design of other contemporary video DiT models (e.g., Sora (Brooks et al., 2024), MovieGen (Polyak et al., 2024), OpenSora (Zheng et al., 2024b), LuminaT2X (Gao et al., 2024a), and CogVideoX (Yang et al., 2024d)). As the backbone, it incorporates a transformer-based architecture with 32 DiT blocks. Each DiT block includes a cross-attention layer for processing text embeddings (produced by the T5-11B model), a self-attention layer, and a fully connected network with a $\times 4$ dimensionality expansion. Attention layers consist of 32 heads with RMSNorm for query and key normalization. Positional information is encoded using 3D RoPE attention, where the temporal, vertical, and horizontal axes are allocated fixed dimensionality within each attention head (using a 2:1:1 ratio). LayerNorm is applied to normalize activations within each DiT block. A pre-trained CogVideoX autoencoder is utilized for video dimensionality reduction, employing causal 3D convolution with a $4\times 8\times 8$ compression rate and 16 channels per latent token. The model features a hidden dimensionality of 4,096 and comprises 11.5B parameters. It leverages block modulations to condition the video backbone on rectified flow timestep information, SiLU activations, and 2×2 ViT-like patchification of input latents to reduce sequence size.

Base video DiT training details. The base DiT model is optimized using AdamW, with a learning rate of 0.0001 and weight decay of 0.01. It is trained for 750,000 iterations with a cosine learning rate scheduler in bfloat16. Image animation support is incorporated by encoding the first frame with the CogVideoX encoder, adding random Gaussian noise (sampled independently from the video noise levels), projecting via a separate learnable ViT-like patchification layer, repeating sequence-wise to match video length, and summing with the video tokens. Training incorporates loss normalization and is conducted jointly on images and videos with variable resolutions (256, 512, and 1024), aspect ratios (16:9 and 9:16 for videos; 16:9, 9:16, and 1:1 for images), and video lengths (ranging from 17 to 385 frames). Videos are generated at 24 frames per second, and variable-FPS training is avoided due to observed performance decreases for target framerates without fine-tuning.

Base video DiT inference details. Inference uses standard rectified flow without stochasticity. We find forty steps to balance quality and sampling speed effectively. For higher resolutions and longer video generation, a time-shifting strategy similar to Lumina-T2X is used, with a time shift of $\sqrt{32}$ for 1024-resolution videos.

C TRAINING DETAILS

Compute details. A single training run for the smaller 700M parameter generator takes approximately 1 day on a node equipped with $8\times$ NVIDIA A100 40GB GPUs, connected via NVIDIA NVLink, along with 960 GB of RAM and 92 Intel Xeon CPUs. The larger 4B parameter model was trained on 8 such nodes for 1.5 days, totaling $64\times$ NVIDIA A100 40GB GPUs. In total, we conducted ≈ 150 training runs for the smaller model during the development stage of 4B generator. Consequently, the project’s total compute utilization amounted to approximately 2700 NVIDIA A100 40GB GPU-days.

Optimization details. We experiment with two model variants: a smaller generator with approximately 700 million parameters for ablations and initial explorations, and a larger 4 billion parameter model, which we use for the main results in this paper. Both models were trained with a batch size of 256 over 50,000 optimization steps with the LAMB optimizer (You et al., 2019). The learning rate was warmed up for the first 10,000 iterations from 0 to 0.005 and then linearly decreased to 0.0015 over subsequent iterations. For the large model, VD3D contains 230M trainable parameters in total which corresponds to around 5% of the total amount. Since the original video diffusion model is trained in the any-frame-conditioning pipeline (Menapace et al., 2024), we can produce variable camera trajectories from the same starting frame. For text conditioning, we use the T5-11B (Raffel et al., 2020) language model to encode text 1024-dimensional embeddings into 128-length sequences. For training efficiency, they were precomputed for the entire dataset. The rest of the training details have been adopted from (Menapace et al., 2024).

Table 3: **Multi-view generation.** We evaluate all models using reference camera trajectories and single-view input images of the RealEstate10K test set. We compute reconstruction metrics based on the subsequent frames for the low-resolution and upsampled high-resolution generations.

Method	Low-resolution			High-resolution		
	PSNR ($\uparrow$)	SSIM ($\uparrow$)	LPIPS ($\downarrow$)	PSNR ($\uparrow$)	SSIM ($\uparrow$)	LPIPS ($\downarrow$)
Base Model	14.74	0.320	0.334	13.23	0.459	0.572
MotionCtrl (U-Net)	15.35	0.387	0.294	13.64	0.473	0.548
CameraCtrl (U-Net)	15.86	0.412	0.266	13.81	0.479	0.540
MotionCtrl	15.07	0.348	0.308	13.42	0.467	0.560
CameraCtrl	14.81	0.327	0.330	13.21	0.456	0.571
Ours	17.23	0.534	0.211	14.90	0.499	0.499
w/o Plucker	14.89	0.346	0.308	13.05	0.455	0.573
w/o ControlNet	14.66	0.313	0.340	13.10	0.450	0.573
w/o weight copy	16.96	0.509	0.220	14.75	0.495	0.504
w/o add context	14.45	0.303	0.368	13.06	0.446	0.588
w/o Plucker context	14.76	0.322	0.318	13.28	0.463	0.579

Table 4: **Quality metrics evaluation.** We evaluate all models using text prompts from the RealEstate10K and MSR-VTT test sets respectively.

Method	RealEstate10K			MSR-VTT		
	FID ($\downarrow$)	FVD ($\downarrow$)	CLIPSIM ($\uparrow$)	FID ($\downarrow$)	FVD ($\downarrow$)	CLIPSIM ($\uparrow$)
Base Model	8.22	160.37	0.2677	3.50	141.26	0.2774
MotionCtrl (U-Net)	2.99	61.70	0.2646	16.85	283.12	0.2411
CameraCtrl (U-Net)	2.48	55.64	0.2681	12.33	201.33	0.2505
MotionCtrl	1.50	52.30	0.2708	9.97	183.57	0.2677
CameraCtrl	2.28	66.31	0.2730	8.47	181.90	0.2690
Ours	1.40	42.43	0.2807	7.80	165.18	0.2689
w/o Plucker	1.17	43.65	0.2715	9.84	152.91	0.2660
w/o ControlNet	3.66	137.06	0.2766	8.34	185.79	0.2674
w/o weight copy	1.38	42.00	0.2710	10.09	218.43	0.2647
w/o add context	1.45	44.74	0.2735	9.56	173.43	0.2657
w/o Plucker context	1.54	43.88	0.2724	8.83	168.95	0.2615

Table 5: **Camera pose evaluation with additional baselines.** We evaluate additional variants of the baselines using reference camera trajectories from the RealEstate10K test set. We compute translation and rotation errors based on estimated camera poses from generations using ParticleSfM (Zhao et al., 2022).

Method	RealEstate10K		MSR-VTT	
	TransError ($\downarrow$)	RotError ($\downarrow$)	TransError ($\downarrow$)	RotError ($\downarrow$)
MotionCtrl (latents)	0.549	0.183	0.650	0.145
CameraCtrl (patches)	0.587	0.197	0.648	0.233
MotionCtrl (frozen)	0.607	0.205	0.678	0.122
Ours	0.409	0.043	0.504	0.050